

Final Project: End-to-End RAG Pipeline with FastAPI

Objective

Build an end-to-end Retrieval-Augmented Generation (RAG) pipeline over a provided dataset. Expose two FastAPI endpoints for retrieval and generation, and design each component for robustness and high-quality outputs. Your system will be evaluated on a hidden test set—aim for accuracy, faithfulness, and consistency.

What You'll Build

- **POST /retrieve:** Returns Top-K most relevant text chunks (with metadata) for a query.
- **POST /generate:** Retrieves context via `/retrieve` and returns an LLM-generated answer.

1. Data Processing (15 pts)

- **Cleaning:** normalize text, strip noise, fix encoding/formatting.
- **Metadata:** attach meaningful tags (e.g., title, date, category).
- **Chunking:** choose fixed-length, sentence/window, or semantic splits; justify size & overlap.
- **Embedding Model:** pick one of the provided models and explain trade-offs.

2. Retrieval Strategy (20 pts)

- **Vector Retrieval:** cosine similarity over chunk embeddings.

- **Hybrid Retrieval:** combine metadata filtering (e.g., by date/category) with vector scores.
- **Top-K Tuning:** allow configurable K; compare results for different values.
- **Justification:** illustrate why your approach yields the best balance of precision/recall.

3. 🧠 Generation Strategy (20 pts)

- **LLM Selection:** choose one provided model or equivalent hosted version.
- **Prompt Design:** include system instructions, context injection, and question formatting.
- **Parameters:** set temperature, max-tokens, top-p, etc.; justify your choices.

4. 🌐 FastAPI Integration (10 pts)

- Two endpoints (`/retrieve`, `/generate`) with clear JSON schemas.
- Modular code structure (e.g., separate `data_utils.py`, `retrieval.py`, `generation.py`).
- Graceful error handling and minimal local setup.

5. 📊 Final Evaluation (35 pts)

Instructor-run evaluation on a hidden test set. Criteria include:

- **Accuracy & Relevance** of answers
- **Faithfulness** (no hallucinations)
- **Coverage** of retrieved chunks
- **Response Consistency** (format and style)
- **Robustness** (edge-case handling)

+ Bonus: Simple UI (+10 pts)

Build a minimal front end (Streamlit, Gradio, or HTML/JS) that:

1. Takes a user query
2. Calls `/generate`
3. Displays retrieved chunks and final answer

What to Submit

1. **Codebase** (clean project folder)
2. **README.md** with run instructions
3. **Short Report** (≤ 3 pages/notebook) covering:
 - Design justifications for each component
 - Sample inputs & outputs
 - Any preprocessing/indexing scripts

Provided Resources

- **Starter Kit:** https://github.com/amindadgar/ai-course/tree/main/final_project_starter
- **Dataset:** <https://www.britannica.com/place/France/Population-structure>
- **Embedding Models (choose one)**
 - `sentence-transformers/all-MiniLM-L6-v2`
 - `togethercomputer/m2-bert-80M-8k-retrieval`
- **LLMs (choose one)**
 - `meta-llama/Llama-3.3-70B-Instruct-Turbo-Free`

- `deepseek-ai/DeepSeek-R1-Distill-Llama-70B-free`
- **LLM Access:** Use TogetherAI's API (<https://www.together.ai>; docs: <https://docs.together.ai>)
- **Vector Stores:** FAISS, ChromaDB, Qdrant, or in-memory
- **Optional Frameworks:** LangChain, llama-index, or custom code

Presentation (Required)

Duration: 15 min (incl. ~3 min Q&A)

Prepare slides or a live walkthrough that answers **5–6** of these questions:

1. **Chunking Strategy:** What approach did you use? Why size/overlap choices?
2. **Embedding Model:** Which model & why? Trade-offs in speed vs. quality?
3. **Retrieval Design:** How does hybrid filtering work? Show an example of its impact.
4. **Prompt Evolution:** Share your final prompt and key iterations.
5. **Performance Highlights:** One strong vs. one weak test case—what insights did you gain?
6. **Future Improvements:** If given more time, what would you enhance or fix?

Tips:

- Focus on decisions & insights, not line-by-line code.
- Illustrate with input/output snapshots.
- Watch your time—practice to fit within 15 minutes.